Midterm Report - Bridge

CEG 4913 – Computer Engineering Design Project II

Fall 2026
University of Ottawa

Professor: Rami Abielmona
Teaching Assistants: Ryan Dufour, Hossam Assad

Group #4 - Bridge

Balemba, Gauthier (300322397)

Brin, Aidan (300317544)

Debnath, Somana (300312973)

Mallette, William (300294650)

Pandya, Stuti (300321622)

Veera, Krisha (300291136)

Submission Date: 23/10/2026

Table of Contents

1. Introduction **2**
1.1 Project Description 2
1.2 Goals, Objectives, Scope 2
1.3 Limitations, Issues and Concerns 3
2. System Diagram **4**
3. Design Analysis **5**
3.1 Hardware Components 5
3.1.1 Raspberry Pi & 9-axis Sensor (Body) 5
3.1.2 ESP32 & 9-axis IMU (Gloves) 5
3.2 Software Components 6
3.2.1 Glove Connection and Application Structure 6
3.2.2 Translate Page 7
3.2.3 Learning Page 7
3.2.4 Glove and Calibration Page 7
3.2.5 Machine Learning Data Collection Page 8
3.2.6 Machine Learning Lab Page 8
4. Implemented System Solution **9**
4.1 Body Unit 9
4.2 Gloves 9
4.3 GUI 10
5. Project Scheduling **11**
5.1 Project Scheduling Table 11
5.2 Gantt Chart 12
6. Task Breakdown **13**
7. Division of Labour **14**
8. Obtained Results **15**
Appendix **18**

1. Introduction

1.1 Project Description

This document reports on the progress of the ASL Gesture Translation Glove capstone project at the midpoint of the second term. It builds on the CEG 4912 final report and covers the design changes made since then, the current state of the hardware and software, and the work remaining before the final demonstration. It is intended for both technical and non-technical audiences, including instructors, evaluators, stakeholders, and future developers.

The team is building a prototype wearable system that translates American Sign Language (ASL) into readable text and audible speech in real time. Each glove uses flex sensors along the fingers and a 9-axis inertial measurement unit (IMU) at the wrist to capture finger bends and hand motion. An ESP32 on each glove sends this data over Bluetooth Low Energy (BLE). A web application decodes the data, recognizes the sign, and displays the result, while a chest-mounted body unit will play the translation aloud.

The prototype consists of three subsystems:

1. **Gloves:** Five flex sensors secured in boning tape, with the ESP32, 9-axis IMU, battery, and power switch on a wrist-mounted protoboard. The first glove is functioning, and the second uses the same design.
2. **Body Unit:** A Raspberry Pi 4B running DietPi, a 9-axis sensor, and a USB speaker, mounted to the chest on a weightlifting belt with a custom 3D-printed mount. It receives glove data over BLE and uses a Piper text-to-speech model to speak the output.
3. **Software:** A React and TypeScript web application for glove connection, calibration, translation, learning, data collection, and model testing, with user accounts through Supabase. The application currently connects to the glove directly, and the recognition model will be moved onto the Raspberry Pi in November 2026.

The main design change this term was moving from hand-written recognition rules to recognition learned from recorded data. Since no public dataset captures flex sensor and IMU readings, the team is collecting its own labelled dataset through the application. An interim nearest-neighbour classifier now covers all 24 static letter classes, though accuracy varies for letters that share similar finger bends. A sequence-based model that uses IMU data and covers moving letters and word signs is planned for November 2026.

1.2 Goals, Objectives, Scope

The primary goal remains to develop a functional prototype of an ASL Gesture Translation Glove that converts hand gestures into readable text and audible speech in real time, improving communication accessibility between people who use ASL and those who do not.

To achieve this goal, the project will focus on the following objectives:

- Complete both gloves with flex sensors and a 9-axis IMU on wrist-mounted electronics.
- Build and mount the body unit for BLE reception and speech output.

- Collect a labelled flex sensor and IMU dataset through the web application.
- Replace the rule-based recognizer with a data-driven model covering static letters, moving signs, and words.
- Deploy the model on the Raspberry Pi and synchronize the body unit with both gloves.
- Test and validate the system for accuracy, reliability, and usability.

Scope: The focus is on a working prototype that recognizes a limited set of predefined ASL signs (between 50 and 100 common letters, signs, and gestures). The project does not aim to support the full ASL language, advanced contextual understanding, or commercial-level deployment. Instead, it demonstrates the feasibility of wearable sensor-based sign translation using embedded systems and software integration.

1.3 Limitations, Issues and Concerns

The 9-axis IMU is currently our biggest hardware limitation. Its readings are noisy and inconsistent, largely because of the breadboard connections used during initial testing. As a result, the model relies only on the five flex sensors. Without wrist orientation, several letters are difficult to distinguish, including G/Q, H/U and K/P. Additionally, some soldering mistakes made during assembly also need to be reworked before we can fully trust the sensor readings.

Since the device is a wearable, hardware development has been slower than expected. Almost every test requires someone to wear the glove, so even small changes involve repeatedly putting it on, testing, removing it and making adjustments. For the same reason, deciding how to mount the protoboard on the glove has taken longer to finalize. Testing on a moving hand also strains the wiring, which often causes wires to come loose. These intermittent disconnects add further noise to our data.

On the software side, the ML model is not yet complete, and data collection is taking longer than planned. We currently have only one sample per letter from a single participant, which limits how much the model can learn and how well it can generalize to new users. The recognizer also supports only static letters, so it does not yet handle J, Z or dynamic signs. Since these challenges build on one another, our main concern is the timeline: we need hardware fixes to get clean IMU data, clean data to collect a proper dataset, and a proper dataset to finish the model before the November deadline.

2. System Diagram

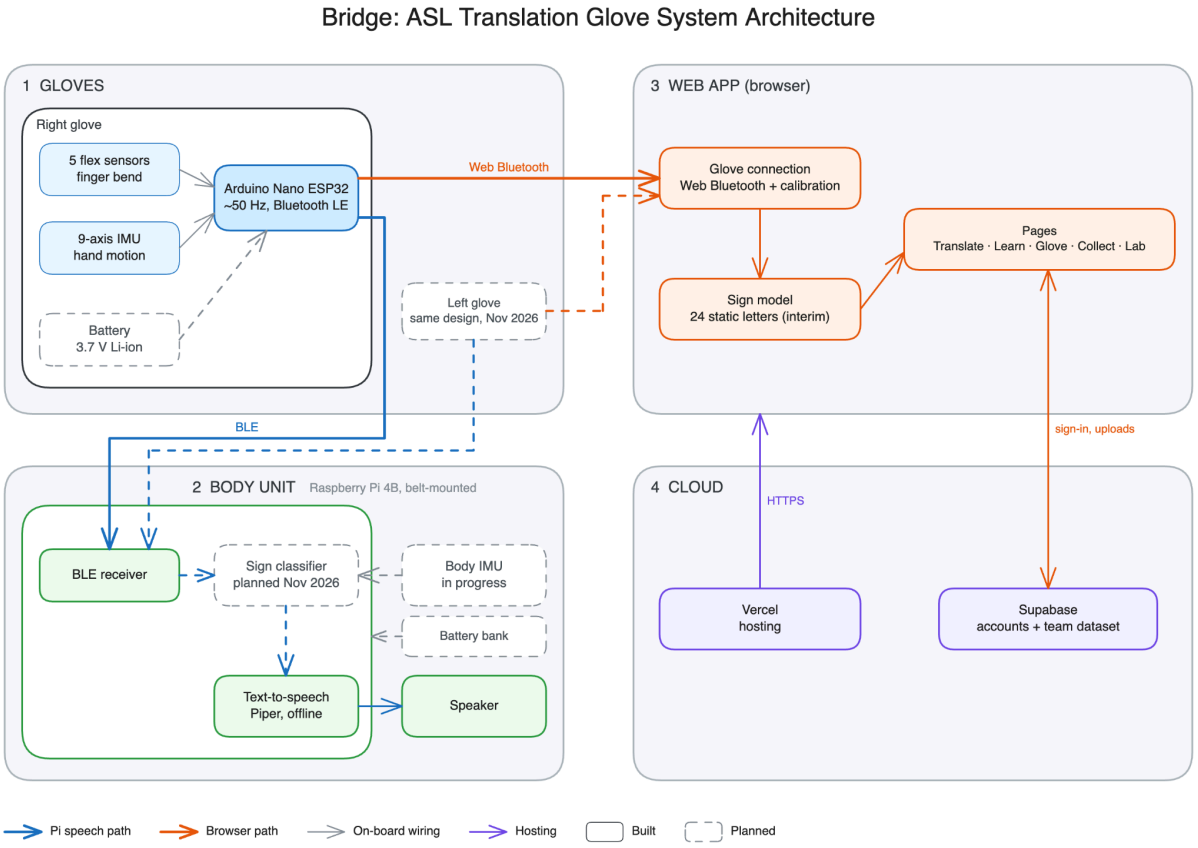


Figure 1: Bridge ASL Translation Glove System Architecture

Figure 1 shows the overall system architecture of Bridge, divided into four main components: the gloves, body unit, web application, and cloud. The gloves use five flex sensors to measure finger bends and a 9-axis IMU to capture hand movements and orientation. An ESP32 on each glove processes the sensor readings and transmits the data over Bluetooth Low Energy (BLE). The web application connects directly to the glove, where the sensor data is used to recognize ASL letters and display the predicted letter on the Translate page. The application also includes features for glove calibration, learning the ASL alphabet, collecting labelled sensor data, and testing the recognition model using a virtual glove.

The body unit uses a Raspberry Pi 4B to receive sensor data and generate audible speech through a connected speaker using the Piper text-to-speech model. On the software side, the web application is hosted on Vercel, while Supabase manages user authentication and dataset storage. At the current stage, the web application uses an interim nearest-neighbour classifier trained on 24 static ASL letter classes, with recognition of moving signs and words still in development. The diagram also distinguishes between implemented components and planned work, highlighting the remaining steps needed to integrate the gloves, body unit, and recognition model into a complete system.

3. Design Analysis

3.1 Hardware Components

The hardware of the ASL Translation System is made up of two gloves and a chest-mounted body unit. Each glove uses five flex sensors to measure finger bends and a 9-axis IMU to capture hand orientation and motion, with an ESP32 reading the sensors and sending the data over Bluetooth Low Energy (BLE). The body unit is built around a Raspberry Pi 4B, which receives the glove data, converts it into text, and plays the result aloud through a speaker. It also carries its own 9-axis sensor to provide a reference point for the position of the gloves.

Since the CEG 4912 final report, both parts of the hardware have changed. On the body unit, the Raspberry Pi now runs DietPi with a streamlined development setup, receives BLE data from the ESP32, and produces speech through a Piper text-to-speech model. On the gloves, the flex sensors are now held in boning tape instead of being sewn on, and the main electronics have moved to a wrist-mounted protoboard after several layout trials. The first glove is functioning, the second is being built to the same design, and the body unit is scheduled to be mounted to a belt using a custom 3D print by early November. The design decisions behind these changes are described below.

3.1.1 Raspberry Pi & 9-axis Sensor (Body)

The body unit has at its center a Raspberry Pi 4B. This system will receive data over BLE from the gloves, convert the raw data it receives into the text of the signed word, and play aloud the generated text as speech over a speaker.

To support rapid prototyping and development, we decided to install a lightweight DietPi operating system and configure the device as a USB gadget and mass storage device to allow for a serial terminal connection and file transfer from a connected computer. This is supported by a simple Python script that checks a SHA-256 hash to detect if the contents have changed, copies the transferred files to a separate directory to avoid read/write concurrency, and runs, if it exists, a Python script called “main.py”.

For Bluetooth reception, the system is using the Bleak library, which limits us to using Python 3.12.10 as the newest version that has a compatible version of Bleak for our platform. For the text-to-speech module, we are using a lightweight Piper model to generate audio from text. The model’s output is at a different sample rate than that of our speaker, so the outputs are resampled to 48 kHz before being played.

The body unit also includes a 9-axis sensor to provide a reference frame for the gloves' position; its data processing is still being finalized.

As for hardware, the Raspberry Pi system, including the 9-axis sensor and speaker, is scheduled to be mounted to a belt using a custom 3D print by early November.

3.1.2 ESP32 & 9-axis IMU (Gloves)

The glove design required several iterations to balance sensor protection, component placement, and hand flexibility. Initially, the flex sensors were sewn directly onto the gloves. However, this did not

secure the sensors sufficiently to maintain their positioning during hand movements. The team researched alternative mounting methods and selected boning tape, a fabric casing commonly used in corsetry. The tape provides a narrow opening for the flex sensors to slide through and can be attached to the glove before the sensors are inserted. This approach protects the sensors while allowing them to be removed or replaced without detaching the tape.

The team also evaluated different methods of assembling the electronic components. A custom printed circuit board (PCB) was initially considered, but manufacturing and delivery could delay development. Its rigid, flat structure could also make it difficult to follow the contours of the hand without restricting movement. The team therefore explored using protoboards, which allow components to be soldered in different arrangements and adjusted throughout development. Working with the lab technician, the team tested several layouts to determine how the components could be positioned without compromising hand mobility.

The initial layout placed the ESP32, 9-axis IMU, and flex sensor connections together on a single board. However, this arrangement restricted movement. The team then experimented with distributing the connections across multiple smaller boards, separating the thumb and pinky from the index, middle, and ring fingers. Although this reduced the concentration of components in one location, the arrangement still did not provide sufficient flexibility and comfort. These trials informed the team's decision to minimize rigid components on the fingers and prioritize a wrist-mounted arrangement for the main electronics.

3.2 Software Components

The front-end of the ASL Translation System is a React and TypeScript web application, built with Vite and Tailwind CSS. It connects to the glove, displays translation results and system status, calibrates the sensors, provides educational content, and, since September 2026, collects labelled training data for the machine learning model.

Since the CEG 4912 final report, the interface has been reorganized into seven pages: Login, Translate, Learn, Glove, Collect, Lab and Account. The Translate page replaced the earlier Dashboard, and the Glove page replaced the separate Sensor and Hand Visualizer pages. Navigation uses client-side routing, and every page except Login requires the user to be signed in. The main design changes made this term and the reasoning behind them are described below.

3.2.1 Glove Connection and Application Structure

In the first prototype, each page managed its own Bluetooth connection to the glove, so moving between pages dropped the connection and the user had to pair again. The connection logic was moved into a single shared component (Sensor Context) that wraps every page of the application. It opens one Web Bluetooth Low Energy (BLE) link to the ESP32, decodes each packet (nine IMU values as 32-bit floats followed by five flex readings as 32-bit integers) and makes the latest reading available to every page. The user connects once and can move between the Translate, Learn, Glove, Collect and Lab pages without reconnecting.

In the current design the browser connects to the glove directly, and the Raspberry Pi is not in this data path. This allowed the interface to be developed and tested against the real glove while the Raspberry Pi was still being set up. The shared component also includes a development-only glove

simulator that generates realistic flex and IMU data at 50 Hz through the same data path as the real glove, so team members can work on the interface without the hardware.

Two further changes support this structure:

- **User accounts:** sign-up and sign-in use Supabase Authentication (email and password). A protected-route wrapper checks for a valid session before any application page loads and sends signed-out users back to the login screen.
- **Saved calibration:** the open-hand and closed-fist reading for each finger is now saved in the browser, so users do not need to recalibrate every time the page reloads. Every page converts raw readings to a calibrated percentage bend with the same function, so values are consistent across the application.

3.2.2 Translate Page

The Translate page, which replaced the earlier Dashboard, is the main translation screen. It shows the sign currently being recognized, and holding a sign for 0.7 seconds types it into the output text. When two letters are too close to separate, the page shows a prompt such as “Not sure: E or S?” instead of guessing. Pause, Space, Delete and Clear controls let the user stop detection or edit the output.

Recognition originally used a rule-based approach that classified each finger as bent or straight. This approach was fast and needed no training data, but a simple bent-or-straight decision cannot separate letters that share the same closed-fist shape, and it recognized only 10 of the 26 letters (A, B, D, F, I, L, S, U, W and Y). It has been replaced by an interim data-driven nearest-neighbour classifier on calibrated flex readings that covers all 24 static letter classes (J and Z are excluded). A model that also uses the IMU and covers word signs is scheduled for November 2026.

3.2.3 Learning Page

The Learn page teaches the 26-letter ASL alphabet. Each letter is shown with a diagram and written instructions on how to form it. With the glove connected, the user can practise each letter and receives per-finger hints when the hand shape does not match, and a quiz tests the letters learned. The page tracks the user’s progress.

3.2.4 Glove and Calibration Page

The Glove page is designed to calibrate and interpret hand gestures using data collected from flex sensors. This calibration process ensures that the system accurately adapts to variations in individual users’ hand movements and finger flexibility.

Upon wearing the glove, users are guided through an initial calibration procedure. They are prompted to fully extend their hand (open position) and then form a closed fist. These two reference positions establish the minimum and maximum sensor values for each finger, allowing the system to determine a personalized range of motion. This calibration improves the accuracy and reliability of subsequent gesture recognition.

Following calibration, the page displays a live hand diagram of the calibrated finger bends, together with the status of the motion sensor (IMU). Letter recognition itself takes place on the Translate page.

Recording of new signs is handled on the Collect page (Section 3.2.5) and through the guided “Teach it your hand” recording on the Lab page.

3.2.5 Machine Learning Data Collection Page

The rule-based recognizer described in Section 3.2.2 could not scale, for three reasons:

- **Lookalike letters:** Some letters differ only in thumb placement, finger spread or wrist orientation (e.g., E/M/N/S/T, U/V/R, G/L/Q).
- **People and gloves differ:** Fixed thresholds do not carry over between users or to a refitted glove.
- **Motion signs:** J, Z and many words involve movement, which cannot be written as a list of finger states.

Data Collection: Data collection is built into the Collect page instead of a separate script. Training samples travel over the same Bluetooth link and processing pipeline as the live recognizer. This prevents a mismatch between the data the model is trained on and the data it sees at runtime.

The first version of the Collect page recorded each static letter for one second (about 50 readings) and averaged them into one row. This smoothed out noise and let the team record the static alphabet in a single session. However, averaging removes the movement that defines moving signs. The Collect page was therefore changed to save every reading as its own row (one row per frame) for all sign types.

Interim Classifier: The first round of data collection produced only one averaged sample per static letter from one participant. That is far too little to train a typical ML model, which needs many examples per letter. Given the time available, nearest-neighbour matching on this dataset was chosen as a baseline. Motion recognition is deferred until the IMU works reliably. Data augmentation is planned for the trained model, since it adds little for nearest-neighbour matching on single samples.

3.2.6 Machine Learning Lab Page

Testing recognition with the physical glove is slow and hard to repeat. Hence, the Lab page is a test bench and simulator where the recognizer can be exercised without hardware. A virtual glove with five sliders sets each finger’s bend and feeds the classifier directly, so any hand shape can be tested. Any recorded letter can be loaded as a starting point, and the user can switch to the live glove when it is available. Wiggle adds random noise (up to a chosen ±%) to each finger of the virtual hand. This simulates sensor noise and natural variation in how a person forms a letter, and shows whether the prediction holds or flips to a neighbouring letter.

The robustness test runs the wiggle automatically. Every recorded sample is jittered many times at the chosen noise level, and the test reports, per letter, how often it is still recognized and what it is mistaken for. With only one sample per letter, test data cannot be held out in the usual way. Noise testing gives a robustness estimate in the meantime and identifies letters that sit too close to a neighbour, which shows which letters to re-record and which will need IMU data. Once a letter has two or more samples, a leave-one-out test classifies each sample using all the others. This gives an honest accuracy figure on real recordings and will replace the noise estimate as the dataset grows.

4. Implemented System Solution

4.1 Body Unit

Beginning with the body unit, and the Raspberry Pi specifically, we successfully installed and made operational a new operating system. This new OS, DietPi, is a lightweight operating system that can run more easily on the Raspberry Pi and allow us to easily manage its other current and future tasks. Following this, we were able to connect our USB speaker to the Raspberry Pi and use a simple Python script to establish a functioning text-to-speech model that we could hear clearly. Our Raspberry Pi is also now able to receive BLE data from the ESP32, and 9-axis motion sensor.

What remains to be done with the body unit is the interpretation of the data coming from the 9-axis motion sensor and refining the data pipeline. For the motion sensor, the one attached to the body will not need to register rotation or speed, just its position in space to find the relative position of the gloves, which reduces the data to interpret here.

With the Raspberry Pi and its 9-axis motion sensor nearing completion, a setup for mounting the Raspberry Pi, the speaker, and the 9-axis sensor to the chest of the user is required. To do this, we purchased a weightlifting belt for the user to wear with the other pieces attached. We chose a weightlifting belt to mount everything as we found that it had large portion of the belt where it became wider and could therefore keep all our required components fixed to the user's chest. This style of belt is also more comfortable to wear around the chest than a regular belt.

To mount the hardware to the belt, we are using the 3D model designing software SolidWorks (2025), as it is a modern piece of software that is freely accessible as uOttawa students, through the on-campus computers or the uOlabsPlus portal. After completing the design, we will be taking advantage of the services provided by the Makerspace on campus to 3D print our mount design.

4.2 Gloves

The current glove design uses boning tape to secure the flex sensors along the fingers while protecting them from damage. The flex sensors and 9-axis IMU on the first glove are functioning; the second glove uses the same design and is scheduled for November 2026. The selected electronics layout places the ESP32 and 9-axis IMU on a protoboard mounted at the wrist, with individual flex sensor connections extending to the fingers. Each flex sensor uses a 10 kΩ resistor and has power, ground, and signal connections. This configuration is intended to reduce restrictions on finger movement while keeping the main electronic components accessible.

The remaining work is to complete the wrist-mounted protoboard by integrating the ESP32, 9-axis IMU, battery, and power switch. The team will then test the assembled hardware to verify sensor readings, electrical connections, and freedom of movement during signing. Once the design has been finalized and tested on one hand, the same configuration will be replicated for the other glove. This work is scheduled for completion in early November 2026.

4.3 GUI

Data collection (Collect page): Each session starts with a short calibration: the participant holds an open hand, then a closed fist, and those readings set the 0% and 100% bend for that person. The user then picks a label and a sign type (static or moving). Every frame in the recording window is saved with its raw flex values, its bend percentages and its IMU values. Samples are flagged if readings arrived too slowly or the IMU sent only zeros. Everything autosaves in the browser and exports as a CSV file.

Letter recognizer and Lab: The recognizer stores a list of labelled handshapes, each with five finger bends from 0 to 1, plus two thresholds: a reject distance and an unsure confidence. It starts from a built-in dataset of 24 static letters recorded with the earlier, averaged version of the Collect page. The recognizer finds the closest stored sample and outputs its letter.

The Lab page is the test bench for this model. A virtual glove with five sliders replaces the real glove; it can load any recorded letter and add random noise ("wiggle"). Training samples can be added, removed, imported or exported. A robustness test copies each sample 40 times with random noise (1–30%) and reports, per letter, how often it is still recognized and what it is mistaken for. Once a letter has two or more samples, a leave-one-out score shows accuracy on the real recordings.

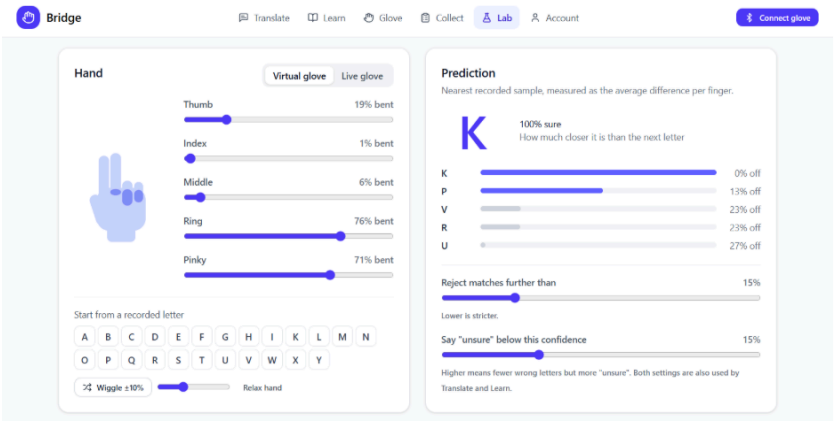


Figure 2: Virtual Glove and Gesture Recognition Interface

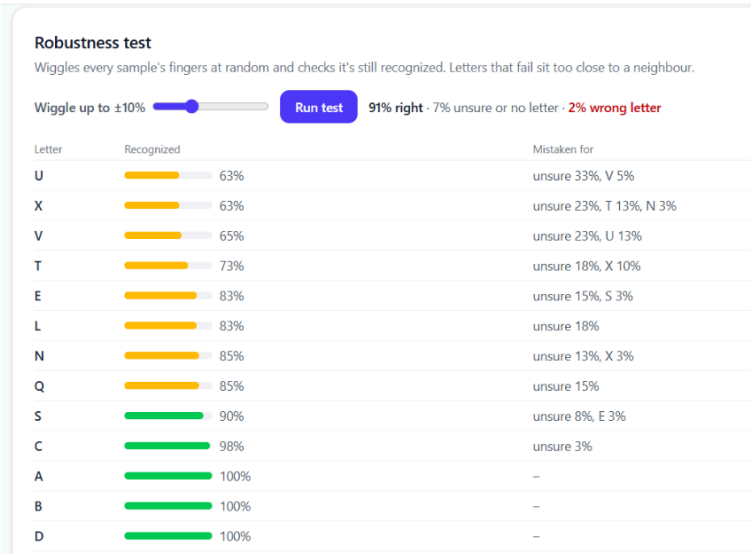


Figure 3: Robustness Test

5. Project Scheduling

5.1 Project Scheduling Table

Task	Start Date or Projected	Status
Finalize scope & requirements	January 2026	Completed
Project Proposal	January 2026	Completed
Order and receive equipment	February 2026	Completed, funding available by EEF for emergency purchases
UI development	February 2026	Completed
Flex sensor Testing	February 2026	Completed
9-axis sensor Testing	February 2026	Completed
Raspberry Pi Testing	February 2026	Completed
Database Setup	March 2026	Completed
Flex sensor to ESP32 connection	March 2026	Completed
Glove Design + Wiring	March 2026	Design Completed, wiring for 2nd glove in progress
Glove Testing	April 2026	Completed for one glove; second glove in progress.
ML model Selection + Training	September 2026	In progress
GUI changes if needed	September 2026	In progress
Building 2nd Glove	September 2026	In progress
9-axis connection to Flex sensors	September 2026	In progress
Body Unit Design	October 2026	In progress
Deploy ML model on Raspberry Pi	November 2026	To be completed
Synchronize Body Unit & Gloves	November 2026	To be completed
Assembling Body Unit	November 2026	To be completed
Documentation & final demo prep	November 2026	To be completed

Table 1: Project Scheduling

5.2 Gantt Chart

The original Gantt charts (Figures 4 and 5) were created in January 2026. Figure 6 reflects the updated schedule as of mid-September 2026. Below is the expected Gantt Chart for January – April and September – December.

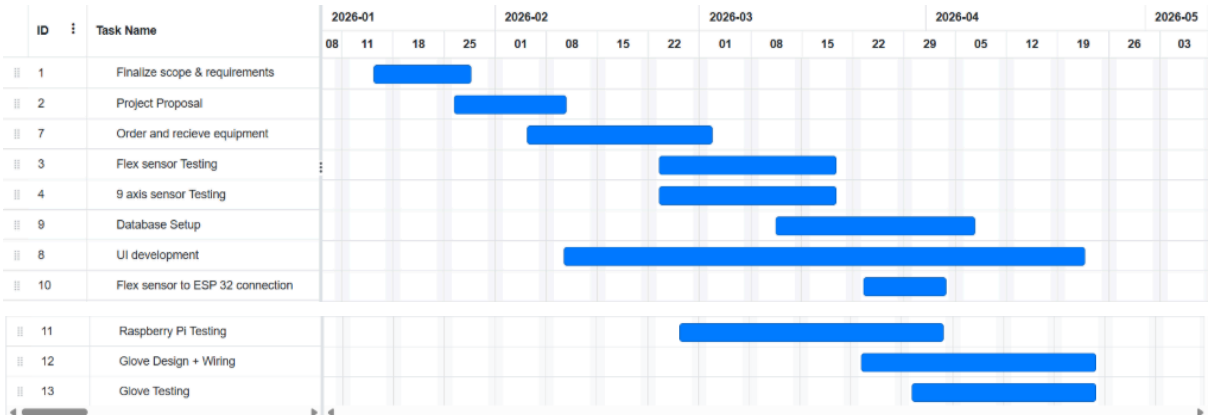


Figure 4: Gantt Chart January-April 2026

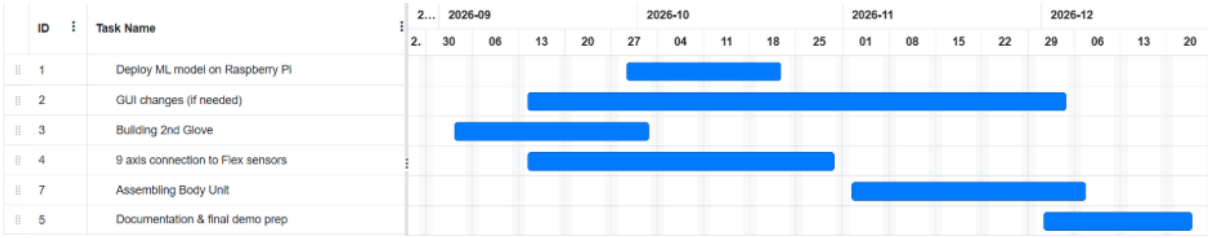


Figure 5: Gantt Chart September-December 2026

A more updated Gantt Chart for September - October was designed in mid-September 2026. It is shown below:

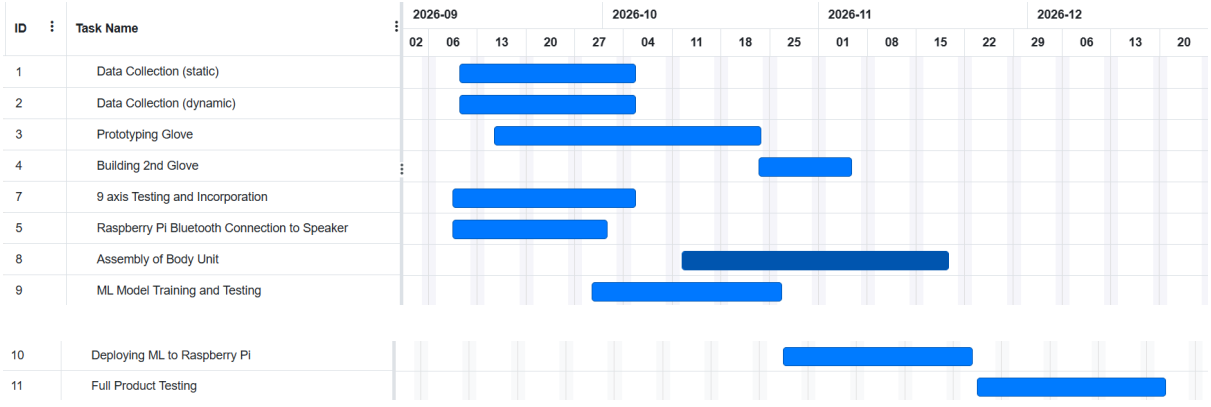


Figure 6: Updated Gantt Chart September-December 2026

6. Task Breakdown

Hardware – Body Unit	Hardware - Gloves	Software - UI and ML
Done: • Operating system and script setup • BLE connection to gloves • Text-to-speech • Speaker testing • 9-axis sensor (in progress) **To-Do:** • Body unit design • Body unit assembly • Full data processing pipeline • ML model deployment	**Done:** • Secured flex sensors along the fingers using boning tape • Tested resistor values and selected 10 kΩ resistors • Connected flex sensors to the ESP32 on the first glove • Verified finger-bending sensor readings • Completed the first glove's flex sensor protoboard (5) • Determined layout of flex sensors & hardware on glove for user comfortability **To-Do:** • Finalize the ESP32 and 9-axis IMU protoboard • Integrate the battery and power switch • Test sensor readings and electrical connections • Verify freedom of movement during signing • Replicate the finalized design for the second glove • Test both gloves together to ensure all values are correctly transferred	**Done:** • Built the Collect page to record labelled glove data • Added calibration at the start of each session (open hand and fist) • Recorded the first dataset: 24 static letters • Added left/right hand columns, sign type and CSV export • Built the letter recognizer from the static dataset • Built the Lab page: virtual glove, wiggle, robustness test, leave-one-out. **To-Do:** • Re-enable the IMU in the firmware • Collect the full dataset with the IMU working • Rebuild the baseline recognizer on the new data • Add IMU features to separate lookalike letters • Train model to include static signs and moving signs • Test on people who weren't in the training data

Table 2: Task Breakdown

7. Division of Labour

Aidan	• Hardware design • Establishing list of 50 to 100 words/phrases to be interpreted • Sensor Calibration • Upcoming work: establishing communication between the computer and the Raspberry Pi • Raspberry Pi chest mount design
Gauthier	• Hardware design • Set up Raspberry Pi Operating System • Capture data from 9-axis sensor • Upcoming work: Establish a connection between the Raspberry Pi and ESP32, Capturing data from ESP32
Krisha	• Designed the wiring layout for the glove and assembled the flex sensors using boning tape • Rebuilt the data collection page with frame-level recording, left/right hand support, sign type selection, quality checks for low frame rate and missing IMU data, IndexedDB autosave, and CSV export. • Collected and organized the initial labelled dataset used to train the nearest-neighbour classifier
Somana	• Calibrated flex sensors & reading flex sensor data • Submitted Engineering Endowment Fund Application for Fall 2026 • Hardware – Glove – Created final working prototype using protoboards for flex sensors, ESP32 & IMU • Hardware – IMU sensor – trial & tested code to pick up values from sensor; and how we can translate those values for our system
Stuti	• Reorganized the web app from three pages to seven (Login, Translate, Learn, Glove, Collect, Lab, Account) • Built the Translate page with nearest-neighbour recognition, 0.7 s hold-to-type, and "unsure" detection for ambiguous letters • Rebuilt the Learn page with per-finger practice hints and a quiz mode • Built the Lab page: virtual glove sliders, wiggle noise simulation, robustness test, and leave-one-out evaluation • IMU and sensor calibration
William	• Sensor calibration • Transmission of ESP32 sensor data over BLE • 9-axis sensor testing and validation • Raspberry Pi Bluetooth module • Raspberry Pi text-to-speech module • Integration testing with the Raspberry Pi

Table 3: Division of Labour

8. Obtained Results

This section summarizes the testing completed so far. Table 4 compares each software requirement against its test and expected result, Table 5 covers the hardware components, and Table 6 lists the current implementation status of each software requirement.

Software Requirement	Description	Test	Expected Result	Obtained Result
GUI Navigation	User can navigate between the main screen and learning tab.	Click between tabs and verify transitions occur correctly.	All tabs load without errors.	Pass
User Authentication	User inputs credentials: system accepts valid logins and rejects invalid ones.	Enter valid and invalid username/password combinations.	Valid credentials grant access; invalid credentials show error message.	Valid sign-in works; invalid-credential test still to be recorded.
ASL Gesture Recognition (ML Model)	System classifies ASL gestures with ≥90% accuracy under controlled conditions.	Perform 10 repetitions of each gesture and record recognition rate.	≥90% of gestures are correctly classified.	Robustness test only (±10% random finger noise, 1 sample per letter from 1 person): 90% correct, 7% unsure, 3% wrong. This is not real-world accuracy; the ≥90% 10-repetition test is still pending.
Wireless Data Transmission	Sensor data is transmitted from the ESP32 to the Raspberry Pi via Bluetooth in real time.	Stream sensor data while performing hand movements and monitor for dropped packets.	Data received continuously with no significant latency or packet loss.	Data received – awaiting finalized glove hardware for accuracy testing
Translation Display	Translated text appears on the GUI within 5–10 seconds of gesture completion.	Perform a gesture and measure time until text appears on screen.	Output displayed within the 5–10 second performance requirement.	≈1.05 s from forming a sign to text, by design (shown after ≈0.35 s, typed after a 0.7 s hold); within the 5–10 s requirement. Stopwatch test not yet recorded.

Software Requirement	Description	Test	Expected Result	Obtained Result
Speech Output	Translated text is spoken aloud through the connected USB speaker.	Trigger a recognized gesture and confirm audio plays through speaker.	Correct English phrase is spoken clearly without distortion.	Text-to-speech implemented – awaiting finalized hardware and sign classification system

Table 4: Software Requirements and Testing Results

Hardware Component	Test	Expected Result	Obtained Result
Flex Sensor 4.5" (x5 per glove)	Bend each finger and read via ESP32 ADC	Resistance increases with bend, distinct repeatable values	Measurable difference in ADC values depending on bend; exact resistance values unknown
ICM-20948 IMU (Glove)	Rotate and tilt glove, read 9-axis outputs	Consistent data matching physical orientation	In progress
ICM-20948 IMU (Body)	Mount on body and compare to glove IMU	Stable reference frame, accurate relative orientation	In progress
Arduino Nano ESP32	Upload firmware, read sensors, transmit over BLE	Streams continuously with no crashes for 5 minutes	Stable connection observed, formal 5-minute test not yet recorded
Raspberry Pi 4B	Boot, receive BLE, run classifier, output to GUI and speaker	Full pipeline without errors within 5–10 s	Not yet implemented
3.7V Li-ion Battery (2500 mAh)	Power ESP32 from battery only	Uninterrupted operation, no thermal issues	Not yet implemented
Raspberry Pi Power	Power Pi from battery only	Stable operation, no overheating	Planning to use USB battery bank instead; not yet implemented
Mini USB Speaker	Connect to Pi, play TTS at various volumes	Clear, intelligible audio	Speaker requires 48 kHz resampling; TTS functional and audio is clear

Hardware Component	Test	Expected Result	Obtained Result
Glove (Full Assembly)	Wear glove, perform 10 ASL gestures	Unique repeatable sensor signatures, under 500g	Not yet implemented

Table 5: Hardware Testing Results

Software Requirement	Description	Implementation Status
Wireless Sensor Communication	System receives gesture data wirelessly from the ESP32 via Bluetooth in real time	Working; full testing awaits finalized glove hardware
Gesture Data Processing	System processes sensor data (flex + IMU) to determine the ASL gesture being performed	Partially implemented; static letters via nearest-neighbour, sequence model scheduled for November 2026
Gesture Data Storage	System stores motion data associated with supported ASL gestures	Done; averaged dataset stored in Supabase, frame-level recordings stored in browser (IndexedDB) with CSV export
ASL Phrase Recognition	System recognizes and translates a predefined set of ~75 ASL signs into English text	Not yet implemented; awaiting ML model and data collection
Graphical User Interface	System displays translated English text through a GUI accessible on a connected device	Implemented; seven pages deployed
Speech Output	System converts translated text into audible speech via the Raspberry Pi and USB speaker	Partially implemented; TTS functional, awaiting translation system
User Authentication	Users can create accounts and log in; system verifies credentials before granting access	Done; Supabase email/password with protected pages
System Logging	System logs recognized gestures and their translations for debugging and evaluation	Scheduled for integration phase

Table 6: Software Implementation Status

Appendix

GitHub Repository: https://github.com/stupnd/Bridge

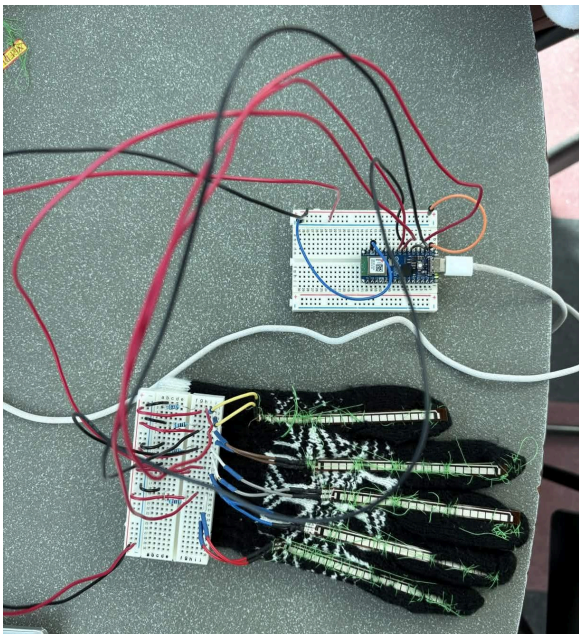


Figure 7: Original Glove Design (January-April 2026)

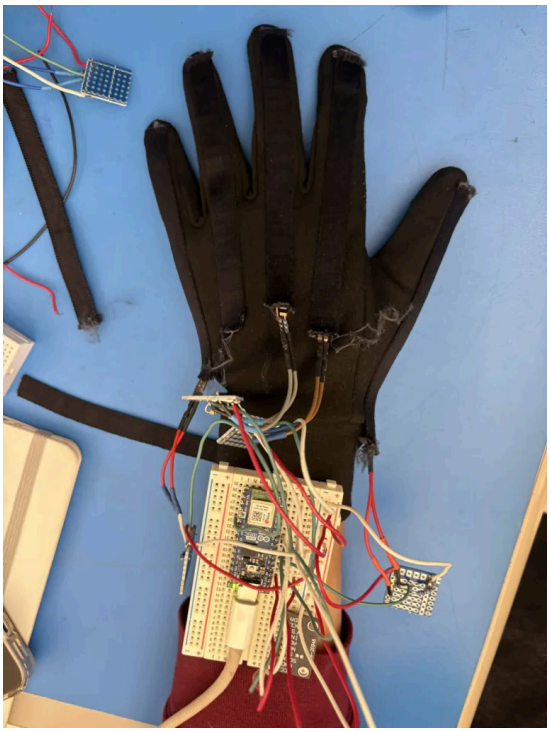


Figure 8: Current Glove with Clothing Bindings to Protect the Flex Sensors

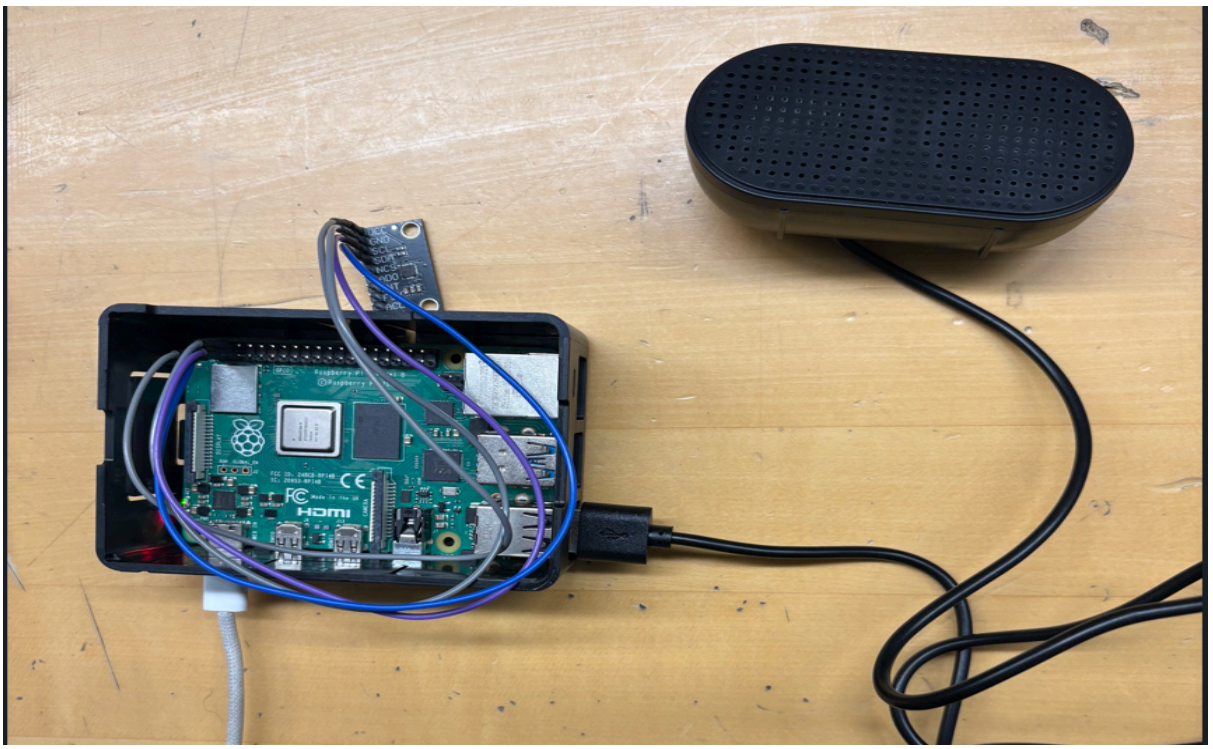


Figure 9: Raspberry Pi 4B Body Unit Connected to the USB Speaker

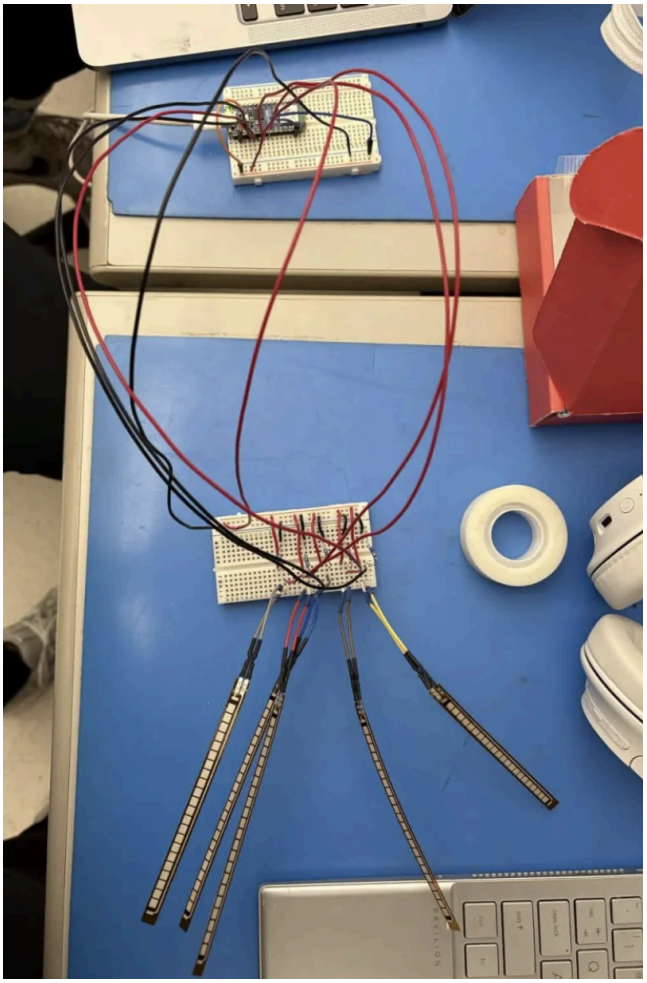


Figure 10: Flex Sensors Wired to the Breadboard and ESP32 for Testing